AutoScaling Ec2 Instances based on CPU usage using Terraform

Published May 13, 2023 • 4 min read • View as Markdown



sunil kumar
I am on mission to build a devOps community, so please be part of it and connect with me over linkedin : https://www.linkedin.com/in/sunilkumar2807/

In this Article we will explore How to scale in and scale down EC2 instances based on there CPU utilization using Autoscaling and Loadbalancer

The source Code is available in

Contents

github.com/Jaffarterraform786/cloudwatch

you can just insert your access key and secret key to Run the Code.

Auto Scaling Groups :

Autoscaling Group also allows you to dynamically control your server pool size — increase it when your web servers are processing more traffic or tasks than usual, or we can decrease or Terminate Instances when Traffic is very less.

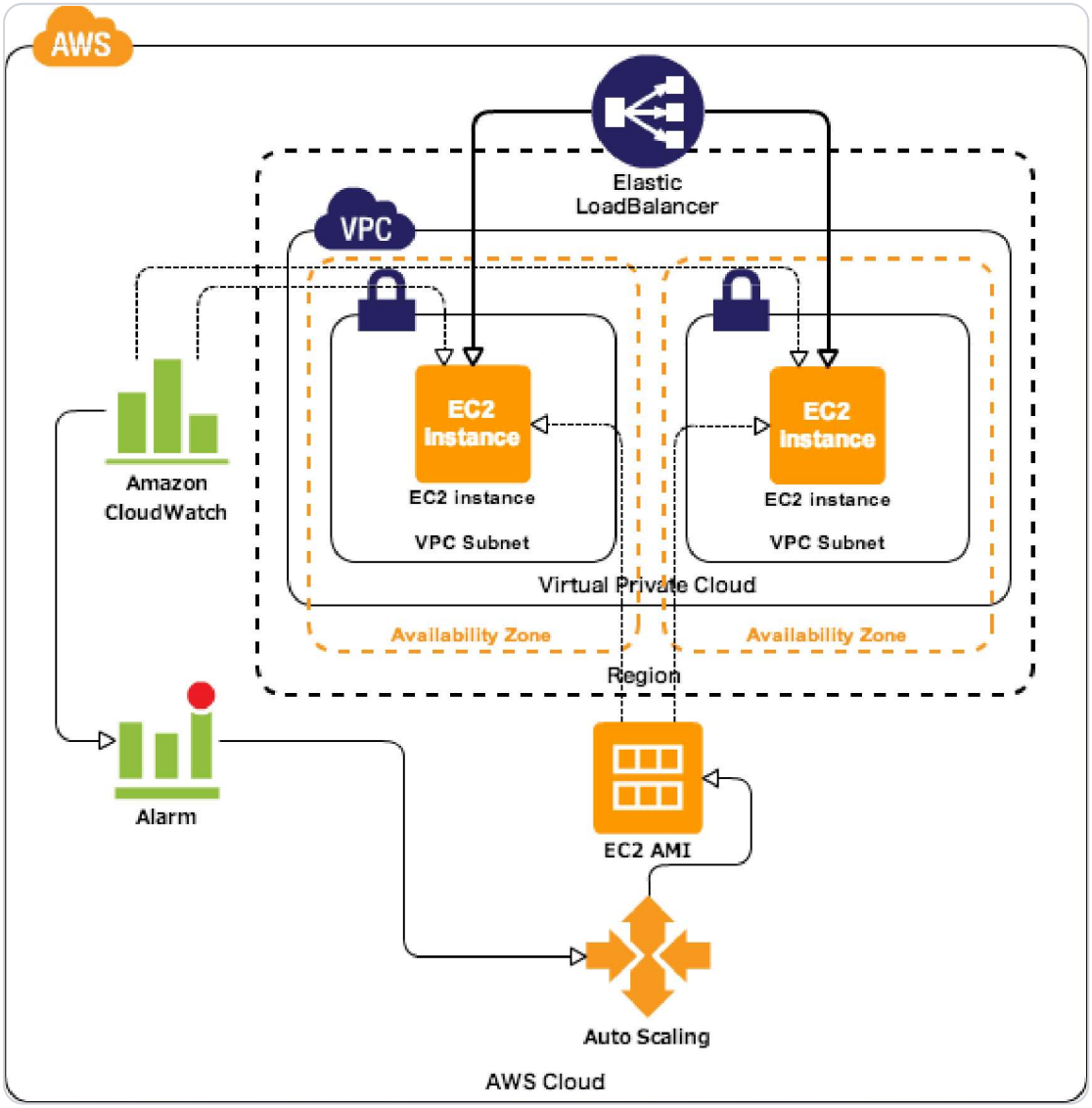


Figure :AutoScaling

Launch Configuration:

AWS Launch Configuration is a template containing all instance settings to apply to each new launched by Auto Scaling Group instance.

```
resource "aws_launch_configuration" "web" {
  name_prefix = "web-"

  image_id = "ami-09662e4f2b2fb67f9" # Amazon Linux 2 AMI (HVM), SSD Volume Type
  instance_type = "t2.micro"
  key_name = "rr"

  security_groups = [ aws_security_group.allow_http.id ]
  associate_public_ip_address = true

  user_data = <<USER_DATA
#!/bin/bash
yum update
sudo yum install httpd -y
    sudo service httpd on
    sudo service httpd start
  USER_DATA

  lifecycle {
    create_before_destroy = true
  }
}
```

Figure : Aws Launch Configuration

Lets create an Loadbalancer ELB .

A Loadbalancer basically Receives the traffic from clients, shuffles the traffic and sends the traffic to the right instance.

USER_DATA:

The `user-data` option is filled with a simple bash-script, which installs the apache web server and puts the instance's local IP address to the index.html file, so we can see it after the instance is up and running.

LIFECYCLE

Lifecycle is a special instruction, which is used in declaring how new launch configuration rules applied during update. We're using **CREATE_BEFORE_DESTROY** here to create new instances from a new launch configuration before destroying the old ones. This option used during rolling deployments..

Lets specify health_check configuration, which determines when Load Balancer should transition instances from healthy to unhealthy state and back depending on its ability to reach HTTP port 80 on the target instance.

Lets specify 2 Subnets, where our Load Balancer will look for (`listener` configuration) launched instances and turned on **CROSS *ZONE* *LOAD*BALANCING** feature, so we could have our instances in different Availability Zones.

```
resource "aws_elb" "web_elb" {
  name = "web-elb"
  security_groups = [
    aws_security_group.elb_http.id
  ]
  subnets = [
    aws_subnet.public_us_east_1a.id,
    aws_subnet.public_us_east_1b.id
  ]

  cross_zone_load_balancing   = true

  health_check {
    healthy_threshold = 2
    unhealthy_threshold = 2
    timeout = 3
    interval = 30
    target = "HTTP:80/"
  }

  listener {
    lb_port = 80
    lb_protocol = "http"
    instance_port = "80"
    instance_protocol = "http"
  }

}
```

Figure ELB

Lets define a security Group

SECURITY GROUP:

1. A security group acts a layer of security around EC2 instances.
2. we can create 5 security groups for an instance.
3. we can specify only allow rules , not deny rules.

```
resource "aws_security_group" "elb_http" {
  name        = "elb_http"
  description = "Allow HTTP traffic to instances through Elastic Load Balancer"
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port       = 0
    to_port         = 0
    protocol        = "-1"
    cidr_blocks     = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Allow HTTP through ELB Security Group"
  }
}
```

Figure security group

Lets launch autoscaling group:

- There will be minimum one instance to serve the traffic.

- Auto Scaling Group will be launched with 2 instances and put each of them in separate Availability Zones in different Subnets.
- Auto Scaling Group will get information about instance availability from the `ELB`.
- We're set up collection for some Cloud Watch metrics to monitor our Auto Scaling Group state.
- Each instance launched from this Auto Scaling Group will have `Name` tag set to `web`.

```
resource "aws_autoscaling_group" "web" {
  name = "${aws_launch_configuration.web.name}-asg"

  min_size             = 1
  desired_capacity     = 2
  max_size             = 4

  health_check_type    = "ELB"
  load_balancers = [
    aws_elb.web_elb.id
  ]

  launch_configuration = aws_launch_configuration.web.name

  enabled_metrics = [
    "GroupMinSize",
    "GroupMaxSize",
    "GroupDesiredCapacity",
    "GroupInServiceInstances",
    "GroupTotalInstances"
  ]

  metrics_granularity = "1Minute"

  vpc_zone_identifier  = [
    aws_subnet.public_us_east_1a.id,
    aws_subnet.public_us_east_1b.id
  ]
```

Figure autoscaling Group

Auto scaling policies:

> *`aws_autoscaling_policy` defines how AWS should change Auto Scaling Group instances count in case of `aws_cloudwatch_metric_alarm`.*
>
> *`cooldown` option is needed to give our infrastructure some time (300 seconds) before increasing Auto Scaling Group again.*

```
resource "aws_autoscaling_policy" "web_policy_up" {
  name = "web_policy_up"
  scaling_adjustment = 1
  adjustment_type = "ChangeInCapacity"
  cooldown = 300
  autoscaling_group_name = aws_autoscaling_group.web.name
}

resource "aws_cloudwatch_metric_alarm" "web_cpu_alarm_up" {
  alarm_name = "web_cpu_alarm_up"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods = "2"
  metric_name = "CPUUtilization"
  namespace = "AWS/EC2"
  period = "120"
  statistic = "Average"
  threshold = "60"

  dimensions = {
    AutoScalingGroupName = aws_autoscaling_group.web.name
  }

  alarm_description = "This metric monitor EC2 instance CPU utilization"
  alarm_actions = [ aws_autoscaling_policy.web_policy_up.arn ]
}
```

Figure Autoscaling policies

> *`aws_cloudwatch_metric_alarm` is a straightforward alarm, which will be fired, if the total CPU utilization of all instances in our Auto Scaling Group is greater or equal `threshold` (60% CPU utilization) during 120 seconds.*

```
resource "aws_cloudwatch_metric_alarm" "web_cpu_alarm_down" {
  alarm_name = "web_cpu_alarm_down"
  comparison_operator = "LessThanOrEqualToThreshold"
  evaluation_periods = "2"
  metric_name = "CPUUtilization"
  namespace = "AWS/EC2"
  period = "120"
  statistic = "Average"
  threshold = "10"

  dimensions = {
    AutoScalingGroupName = aws_autoscaling_group.web.name
  }

  alarm_description = "This metric monitor EC2 instance CPU utilization"
  alarm_actions = [ aws_autoscaling_policy.web_policy_down.arn ]
}

output "elb_dns_name" {
  value = aws_elb.web_elb.dns_name
}
```

Here we're decreasing Auto Scaling Group size by one instance every 300 seconds if its total CPU utilization is less or equals 10%

Lets Do Terraform Plan to provision the resources.

```
Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:

elb_dns_name = "web-elb-89873292.us-east-2.elb.amazonaws.com"

C:\Users\alfau\Desktop\88>terraform state list
aws_autoscaling_group.web
aws_autoscaling_policy.web_policy_down
aws_autoscaling_policy.web_policy_up
aws_cloudwatch_metric_alarm.web_cpu_alarm_down
aws_cloudwatch_metric_alarm.web_cpu_alarm_up
aws_elb.web_elb
aws_internet_gateway.my_vpc_igw
aws_launch_configuration.web
aws_route_table.my_vpc_public
aws_route_table_association.my_vpc_us_east_2a_public
aws_route_table_association.my_vpc_us_east_2b_public
aws_security_group.allow_http
aws_security_group.elb_http
aws_subnet.public_us_east_1a
aws_subnet.public_us_east_1b
aws_vpc.my_vpc
```

Figure Resources Launched.

As the desired capacity is 2 we can see two EC2 instnaces Launched in console.

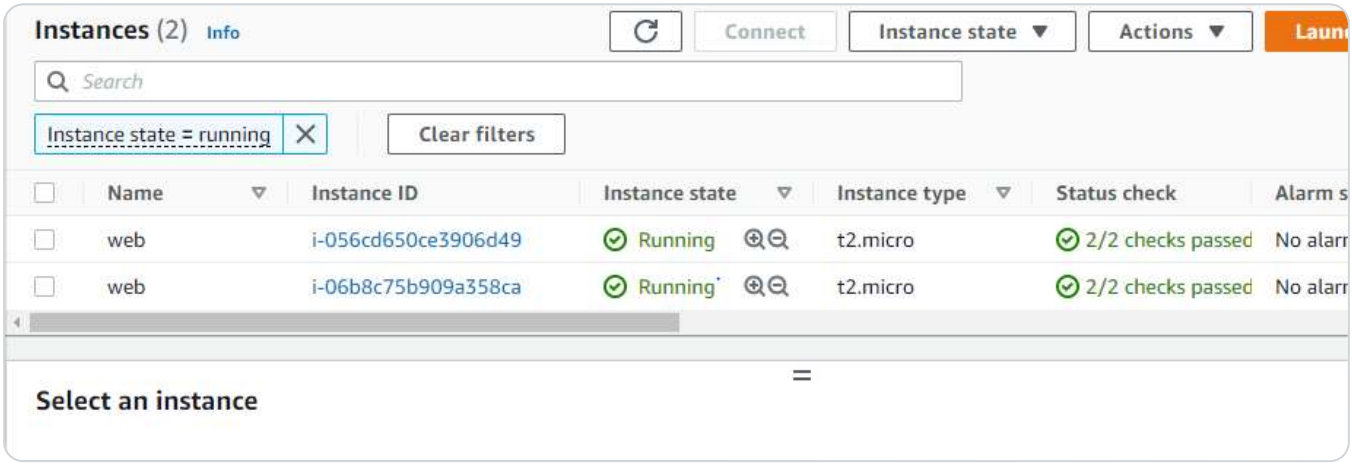


Figure: Initially 2 instances Launched

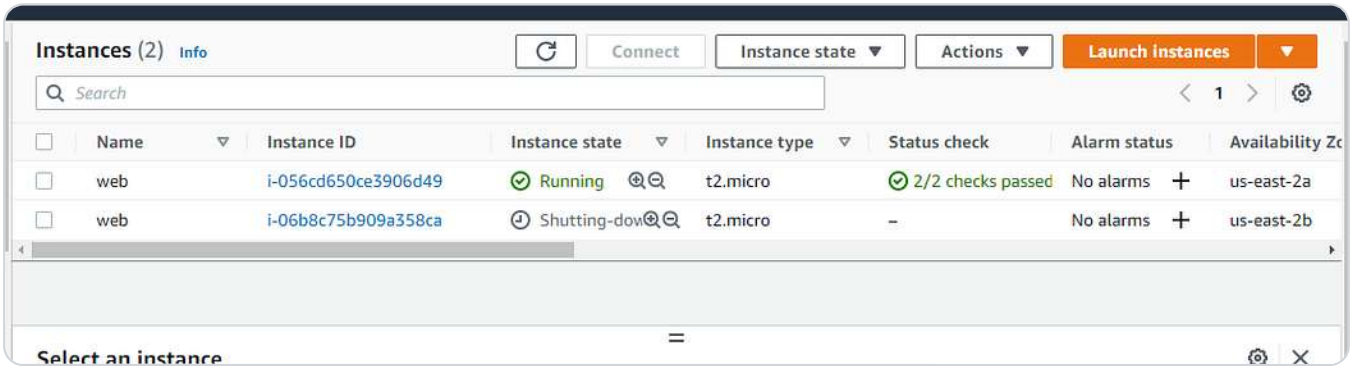


Figure : out of 2 instances 1 shutting Down

We can see 1 instance is shutting down.

This is because of the Cloud Watch alarm , Here we're decreasing Auto Scaling Group size by one instance every 300 seconds if its total CPU utilization is less or equals 10%.

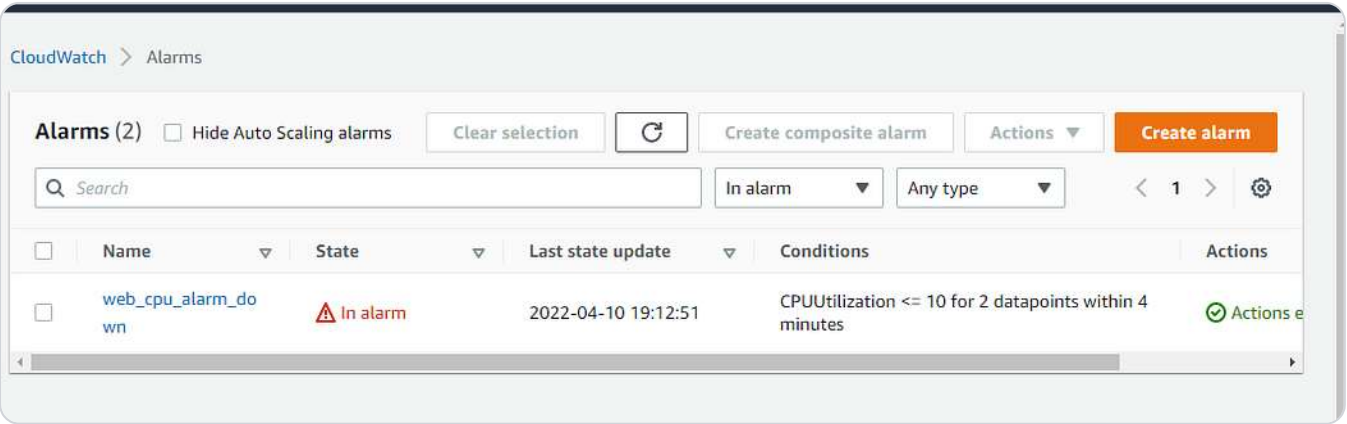


Figure :Alarm that caused Termination of an Instance.

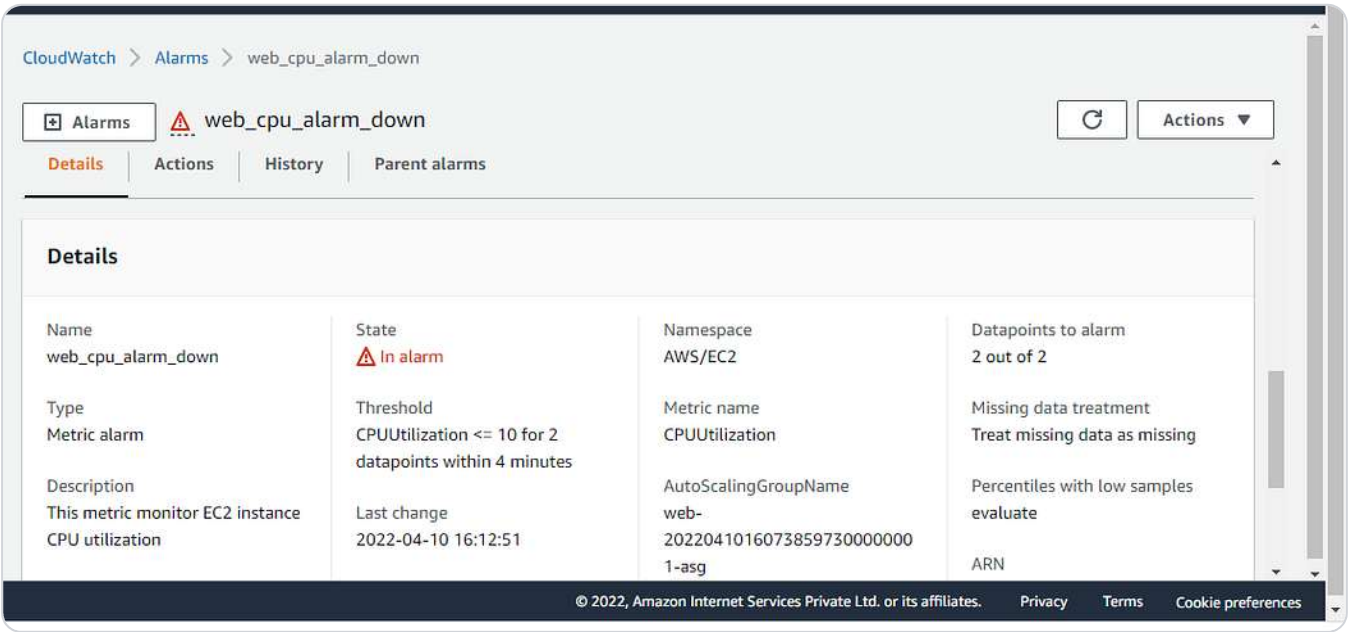


Figure :Cloudwatch Alarm for cpu utilization<10 percent

Conclusion:

In this Article we have seen how to set up a dynamic Auto Scaling Group and Load Balancer to distribute traffic to your instances in several Availability Zones with cloudwatch Metrics.

Credits : jaffer shaikh

I am Sunil kumar, Please do follow me here and support #devOps #trainwithshubham #github #devopscommunity #devops #cloud

#devoparticles #trainwithshubham

sunil kumar

Shubham Londhe

Connect with me over linkedin : linkedin.com/in/sunilkumar2807

#devops #developer #cloud #javascript #aws

Comments (1)

Join the discussion



Sandip Kumar 3y ago

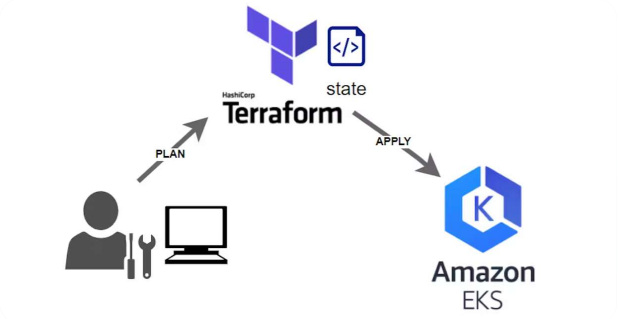
nice

More from this blog

How to Create EKS Cluster Using Terraform?

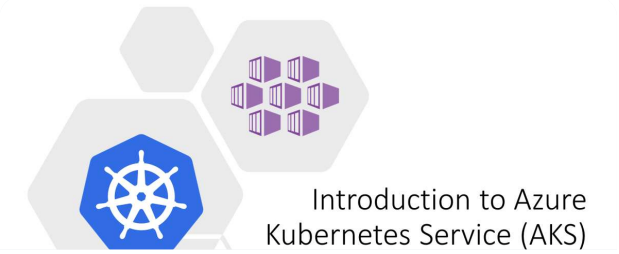
Create AWS VPC using Terraform¶ Let's start with terraform. First, we need to create an AWS provider. It allows to interact with the many resources supported...

Sep 2, 2023 · 13 min read



Azure Kubernetes Service (AKS)

What is Azure Kubernetes Service (AKS)? Azure Kubernetes Service is a managed container orchestration service based on the open source...



Jul 14, 2023 · 7 min read

What is Azure Web App? Create an Azure Web App

Web Apps are one of the most widely used Azure services that enable us to host our web applications in Azure. It can host a User Interactive application or ev...

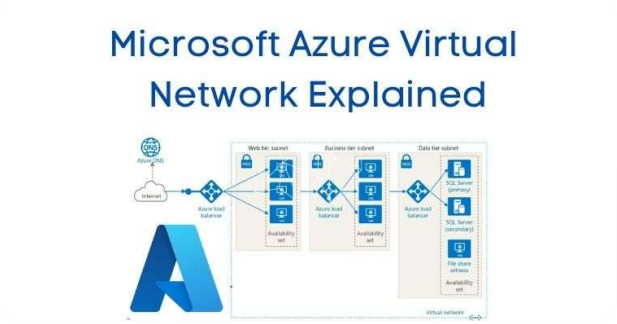
Jul 13, 2023 · 8 min read



What Is Azure Virtual Network?

An Azure Virtual Network (VNet) is a network or environment that can be used to run VMs and applications in the cloud. When it is created, the...

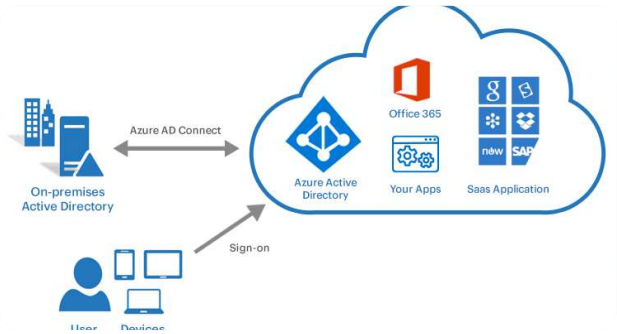
Jul 9, 2023 · 4 min read



What is Azure Active Directory?

Azure Active Directory (Azure AD) is Microsoft's multi-tenant, cloud-based Identity and Access Management (IAM) service. It takes care of authentication and...

Jul 8, 2023 · 10 min read



Sunil Kumar

87 posts

© 2026 Sunil Kumar